Multi-Client Chat Application

Name: Nikhil Yadav
Roll No: 202310101200028
Github Repo : link

---

1. Introduction

This project is a multi-client chat application built using TCP socket programming in Python. It uses a client–server model where many users (clients) connect to one main server and send messages to each other in real time.
The server is responsible for handling all client connections, sending messages to all connected users, and keeping track of who is connected. Each client can send messages, receive messages from others, and leave the chat using a command.
This project helps in understanding important networking concepts such as socket programming, handling multiple users at the same time (concurrency), and communication over a network.

---

2. System Architecture

The application uses a client–server model.

- The server listens for incoming client connections on a specified port.
- Each client connects to the server using the server's IP address and port number.
- Messages sent by a client are received by the server and broadcast to all other connected clients.

Architecture overview:

```
None
           Chat Server
               |
       --------------------------
       |        |         |
    Client1  Client2   Client3 ----
```

The server maintains a list of connected users and forwards messages between them.

---

3. Concurrency Model

To support multiple users simultaneously, the server uses a **thread-per-client model**.
When a client connects:

1. The server accepts the connection.
2. A new thread is created for that client.
3. The thread handles all communication with that client.

This design allows multiple clients to send messages at the same time without blocking the server or other clients.

---

4. Communication Protocol

The application uses a **simple text-based protocol** for communication between clients and the server.

Basic Commands

Command	Description
JOIN	Sent when a client connects with a username
MSG	Regular chat message
/users	Display list of connected users
/quit	Client disconnect request

Example:
[20:14:54] Nikhil : hello
[20:15:01] Online users: Nikhil,Sumit
The server timestamps messages and broadcasts them to all connected clients except the sender.

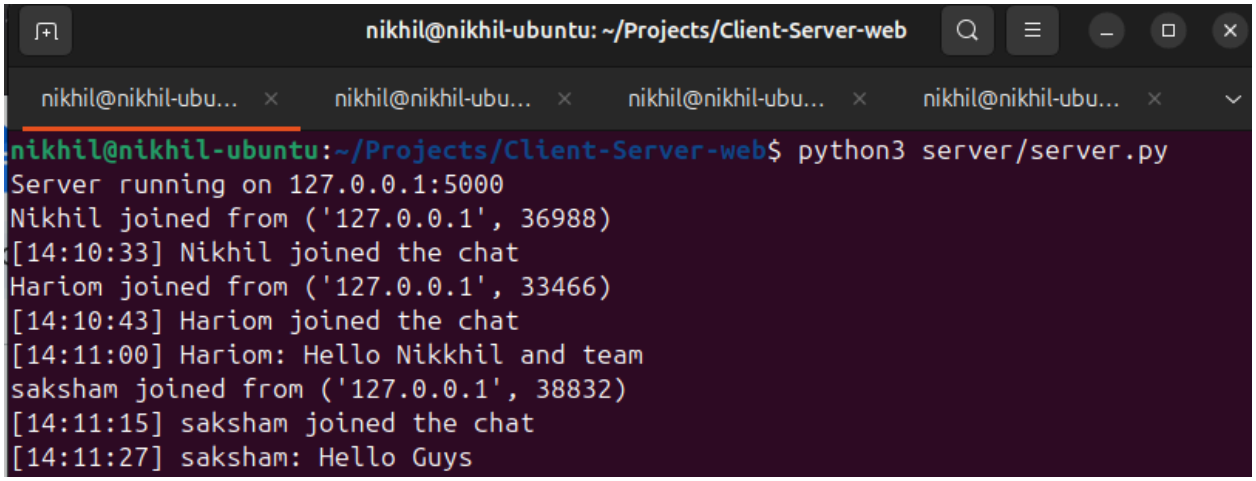
---

5. Testing

The application was tested using multiple terminal windows to simulate several users.

Server Startup and Multiple Clients Connecting

The server starts successfully and listens for client connections and several clients join.



Message Broadcasting, user list, and client disconnection

Messages sent from one client were received by all other connected clients in real time. And also showing the /users command correctly displayed the list of connected users.
And at the end when a client used /quit, the server removed the user and notified the remaining clients.

```
nikhil@nikhil-ubu...   nikhil@nikhil-ubu...   nikhil@nikhil-ubu...   nikhil@nikhil-ubu...
nikhil@nikhil-ubuntu:~/Projects/Client-Server-web$ python3 client/client.py
Enter your username: Nikhil
Connected to chat server
[14:10:33] Nikhil joined the chat
[14:10:43] Hariom joined the chat
[14:11:00] Hariom: Hello Nikkhil and team
[14:11:15] saksham joined the chat
[14:11:27] saksham: Hello Guys
Hello Dear ones
[14:11:55] saksham: Hope u are doing well
[14:11:58] Hariom: YEs
how are you saksham?
[14:55:47] saksham: Yes, I am good
```

New users joined, chat history and then active users and when it quit it disconnected.

```
nikhil@nikh...   nikhil@nikh...   nikhil@nikh...   nikhil@nikh...   nikhil@nikh...
nikhil@nikhil-ubuntu:~/Projects/Client-Server-web$ python3 client/client.py
Enter your username: suarabh
Connected to chat server
---- Recent Chat History ----
[14:10:33] Nikhil joined the chat
[14:10:43] Hariom joined the chat
[14:11:00] Hariom: Hello Nikkhil and team
[14:11:15] saksham joined the chat
[14:11:27] saksham: Hello Guys
[14:11:35] Nikhil: Hello Dear ones
[14:11:55] saksham: Hope u are doing well
[14:11:58] Hariom: YEs
[14:55:38] Nikhil: how are you saksham?
[14:55:47] saksham: Yes, I am good
[14:56:47] Online users: Nikhil, Hariom, saksham
[14:56:54] Online users: Nikhil, Hariom, saksham
-----------------------------

[14:57:08] suarabh joined the chat
/users
[14:57:20] Online users: Nikhil, Hariom, saksham, suarabh
/quit
nikhil@nikhil-ubuntu:~/Projects/Client-Server-web$
```

---

6. Challenges Faced

Several challenges were encountered during the development process, which I'm mentioning below:
Handling Multiple Clients

Initially, the server handled only one client at a time. This issue was resolved by implementing **multi-threading**, allowing multiple clients to connect simultaneously.

Port Already in Use Error
During development, the server occasionally failed to start because the port was already in use. This was solved by enabling the SO_REUSEADDR socket option.

Handling Client Disconnections
Unexpected client disconnections sometimes caused errors in the server. Proper exception handling and client removal from the active connection list resolved this issue.

---

7. Extra Features Implemented

The following enhancements were added beyond the basic requirements:

- Message timestamps
- Persistent chat history stored in chat.log
- Server logging of chat events
- /users command to display connected clients

---

8. Conclusion

This project successfully demonstrates the implementation of a **multi-client chat application using TCP socket programming**. The system supports concurrent clients, real-time message broadcasting, and user commands.
Through this assignment, I explored important and implemented networking concepts such as **client–server communication, concurrency, and protocol design**.